

Setup

In this section, we will set up the computing environment required to perform the analyses for this part. The setup involves using both Shell and R within this notebook.

First, we will create a function called `shell_call` to execute shell scripts directly.

Following that, we will install and configure R to ensure it is ready for use

R is a programming language and software environment for data analysis, statistics, and graphics. It is widely used in scientific research, machine learning, bioinformatics, and data science due to its flexibility and vast collection of packages. R is open source and offers tools for data manipulation, statistical modeling, visualization, and integration with other languages such as Python and C++.

```
In [ ]: # Original Tutorial: https://stackoverflow.com/questions/70025153/how-to-access-
shell_call <- function(command, ...) {
  result <- system(command, intern = TRUE, ...)
  cat(paste0(result, collapse = "\n"))
}

loadPackages = function(pkgs){
  myrequire = function(...){
    suppressWarnings(suppressMessages(suppressPackageStartupMessages(require(...
  })
  ok = sapply(pkgs, require, character.only=TRUE, quietly=TRUE)
  if (!all(ok)){
    message("There are missing packages: ", paste(pkgs[!ok], collapse=", "))
  }
}

# Setup R2U for Ubuntu Jammy

download.file("https://github.com/eddelbuettel/r2u/raw/master/inst/scripts/add_c
              "add_cranapt_jammy.sh")
Sys.chmod("add_cranapt_jammy.sh", "0755")
shell_call("./add_cranapt_jammy.sh")
bspm::enable()
options(bspm.version.check=FALSE)
shell_call("rm add_cranapt_jammy.sh")

# Install required packages
cranPkgs2Install = c("BiocManager", "ggpubr", "Seurat", "cowplot",
                    "Rtsne", "hdf5r", "clustree", "SoupX")

install.packages(cranPkgs2Install, ask=FALSE, update=TRUE, quietly=TRUE)

biocPkgs2Install = c("clusterProfiler", "preprocessCore", "rols",
                    "scDbfFinder", "clusterExperiment", "SingleR",
                    "celldex", "org.Hs.eg.db", "DropletUtils",
                    "miloR", "scran", "scater")
BiocManager::install(biocPkgs2Install, ask=FALSE, update=TRUE, quietly=TRUE)
```

Packages in R

Packages in R

A package in R is a collection of functions, data, and documentation bundled together. Packages help extend the core functionality of R by providing specific tools for different tasks, such as data analysis, graph visualization, advanced statistics, etc.

Install

Before using a package, it must be installed in your environment. This is done only once (unless it needs to be updated) with the command:

```
In [ ]: install.packages("name_of_package")
```

Library

Once installed, the package needs to be loaded in each session in which you want to use it. The command checks whether the mentioned package is installed. If the package is installed, it is loaded and ready for use. If the package is not installed, R returns an error.

This is where the `library()` function comes in:

```
In [ ]: library(name_of_package)
```

```
In [ ]: # To simplify package loading, we created the LoadPackages() function.  
# But, if you don't have the function, you should use 'library(name_of_package)'  
  
pkgs = c("Rtsne", "Seurat", "tidyverse", "cowplot",  
         "scDblFinder", "clustree", "preprocessCore",  
         "clusterProfiler", "celldex", "SoupX",  
         "DropletUtils", "miloR", "scraper", "scater")  
loadPackages(pkgs)
```

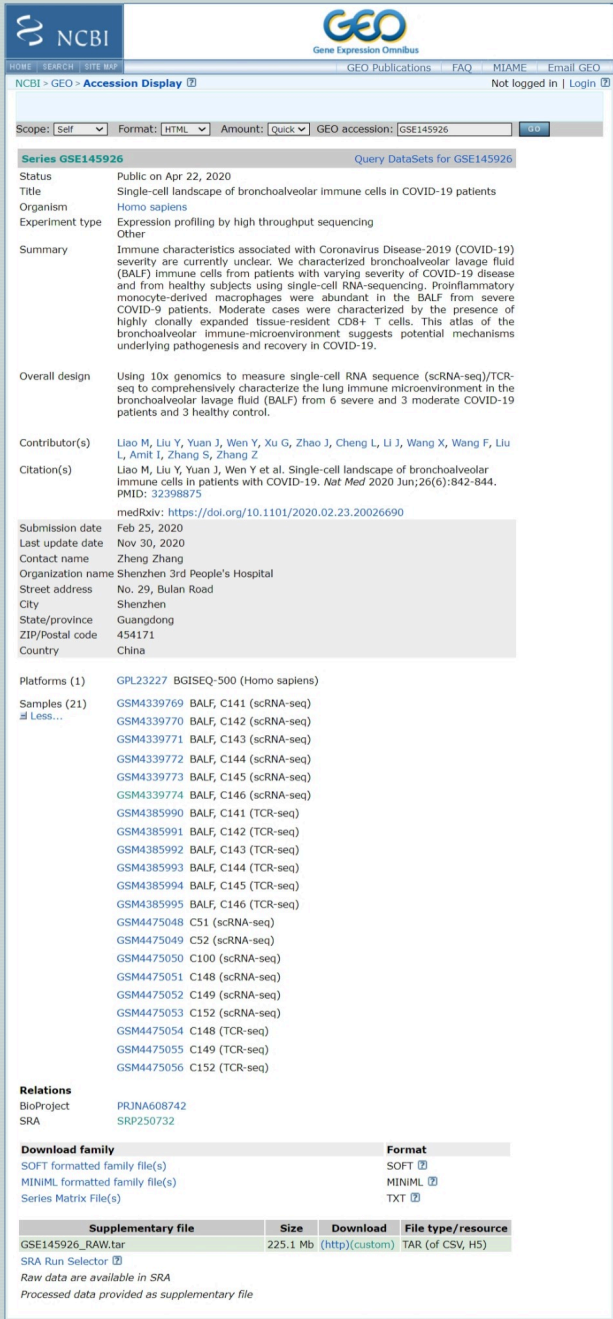
Then, load the libraries that you previously installed.

- **Rtsne:** package in R implements the t-SNE technique for dimensionality reduction and data visualization.
- **Seurat:** Seurat is an R package designed for QC, analysis, and exploration of single cell RNA-seq data.
- **scDblFinder:** The scDblFinder package gathers various methods for the detection and handling of doublets/multiplets in single-cell sequencing data (i.e. multiple cells captured within the same droplet or reaction volume)
- **Cowplot:** Provides various features that help with creating publication-quality figures with `ggplot2`, such as a set of themes, functions to align plots and arrange them into complex compound figures, and functions that make it easy to annotate plots and or mix plots with images.
- **clustree:** This visualization shows the relationships between clusters at multiple resolutions, allowing researchers to see how samples move as the number of clusters increases.

- **PreprocessCore:** A collection of pre-processing functions

Data input, sparse UMI-count matrix

Original Article: [Single-cell landscape of bronchoalveolar immune cells in patients with COVID-19](#)



NCBI GEO Accession Display

Scope: Self Format: HTML Amount: Quick GEO accession: GSE145926

Series GSE145926 Query DataSets for GSE145926

Status: Public on Apr 22, 2020
 Title: Single-cell landscape of bronchoalveolar immune cells in COVID-19 patients
 Organism: Homo sapiens
 Experiment type: Expression profiling by high throughput sequencing
 Other: Other
 Summary: Immune characteristics associated with Coronavirus Disease-2019 (COVID-19) severity are currently unclear. We characterized bronchoalveolar lavage fluid (BALF) immune cells from patients with varying severity of COVID-19 disease and from healthy subjects using single-cell RNA-sequencing. Proinflammatory monocyte-derived macrophages were abundant in the BALF from severe COVID-19 patients. Moderate cases were characterized by the presence of highly clonally expanded tissue-resident CD8+ T cells. This atlas of the bronchoalveolar immune-microenvironment suggests potential mechanisms underlying pathogenesis and recovery in COVID-19.
 Overall design: Using 10x genomics to measure single-cell RNA sequence (scRNA-seq)/TCR-seq to comprehensively characterize the lung immune microenvironment in the bronchoalveolar lavage fluid (BALF) from 6 severe and 3 moderate COVID-19 patients and 3 healthy control.
 Contributor(s): Liao M, Liu Y, Yuan J, Wen Y, Xu G, Zhao J, Cheng L, Li J, Wang X, Wang F, Liu L, Amit I, Zhang S, Zhang Z
 Citation(s): Liao M, Liu Y, Yuan J, Wen Y et al. Single-cell landscape of bronchoalveolar immune cells in patients with COVID-19. *Nat Med* 2020 Jun;26(6):842-844. PMID: 32398875
 medRxiv: <https://doi.org/10.1101/2020.02.23.20026690>
 Submission date: Feb 25, 2020
 Last update date: Nov 30, 2020
 Contact name: Zheng Zhang
 Organization name: Shenzhen 3rd People's Hospital
 Street address: No. 29, Bulan Road
 City: Shenzhen
 State/province: Guangdong
 ZIP/Postal code: 454171
 Country: China
 Platforms (1): GPL23227 BGISEQ-500 (Homo sapiens)
 Samples (21): GSM4339769 BALF, C141 (scRNA-seq)
 GSM4339770 BALF, C142 (scRNA-seq)
 GSM4339771 BALF, C143 (scRNA-seq)
 GSM4339772 BALF, C144 (scRNA-seq)
 GSM4339773 BALF, C145 (scRNA-seq)
 GSM4339774 BALF, C146 (scRNA-seq)
 GSM4385990 BALF, C141 (TCR-seq)
 GSM4385991 BALF, C142 (TCR-seq)
 GSM4385992 BALF, C143 (TCR-seq)
 GSM4385993 BALF, C144 (TCR-seq)
 GSM4385994 BALF, C145 (TCR-seq)
 GSM4385995 BALF, C146 (TCR-seq)
 GSM4475048 C51 (scRNA-seq)
 GSM4475049 C52 (scRNA-seq)
 GSM4475050 C100 (scRNA-seq)
 GSM4475051 C148 (scRNA-seq)
 GSM4475052 C149 (scRNA-seq)
 GSM4475053 C152 (scRNA-seq)
 GSM4475054 C148 (TCR-seq)
 GSM4475055 C149 (TCR-seq)
 GSM4475056 C152 (TCR-seq)
 Relations: BioProject: PRJNA608742
 SRA: SRP250732
 Download family: SOFT formatted family file(s) Format: SOFT
 MINIML formatted family file(s) MINIML
 Series Matrix File(s) TXT
 Supplementary file: GSE145926_RAW.tar 225.1 Mb (http/custom) TAR (of CSV, H5)
 SRA Run Selector
 Raw data are available in SRA
 Processed data provided as supplementary file

NLM | NIH | GEO Help | Disclaimer | Accessibility | HHS Vulnerability Disclosure

Download the GEO's raw data

```
In [ ]: # This command in R downloads a file from the internet and saves it locally
download.file('https://www.ncbi.nlm.nih.gov/geo/download/?acc=GSE145926&format=f

# This command uses the shell_call custom function to execute a shell command.
```

```
# The tar -xf command is used to extract the contents of a compressed file in .t
shell_call("tar -xf GSE145926_RAW.tar")

# rm is a shell command to delete files, to free up space and keep the directory
shell_call("rm GSE145926_RAW.tar")

# ls is used to list files.
# -l: Display file details (permissions, owner, size, date, etc.).
# -h: Format file sizes in a human-readable format, such as KB, MB, or GB.
shell_call("ls -lh")
```

Separate the scRNA and TCRseq files.

```
In [ ]: # Create a directory
shell_call("mkdir -p TCRseq")

# mv is the shell command to move or rename files.
# The pattern *_filtered_contig_annotations.csv.gz uses the wildcard character *
shell_call("mv *_filtered_contig_annotations.csv.gz TCRseq/")

shell_call("mkdir -p scRNAseq")

# This command moves all files that have _filtered_ in their name to the scRNAseq
shell_call("mv *_filtered_* scRNAseq/")
shell_call("ls -lh")
```

We're now going to read data for one sample. The 10X Genomics data is usually stored in the HDF5 format (Hierarchical Data Format version 5) is a file format designed to store and organize large volumes of data efficiently, which is a scientific high-performance format.

The `Read10X_h5()` command is able to import this data and load it into R, using a special type of matrix, which has an efficient way of representing data with lots of zeroes.

How were these files created?

```
In [5]: sc <- Read10X_h5("scRNAseq/GSM4339769_C141_filtered_feature_bc_matrix.h5")
```

```
In [ ]: # Is there anything special about this matrix?
sc[33493:33500,1:3]
```

In single-cell RNA sequencing (scRNA-seq) data, most values in the matrix are zeros (0), representing genes with no detected molecules in many cells. To handle this efficiently, Seurat uses a sparse matrix representation whenever possible.

Sparse matrices store only the non-zero values and their locations, significantly reducing memory usage and improving processing speed for datasets like Drop-seq, inDrop, or 10x Genomics data.

For example, converting a dense matrix (~1600MB) to a sparse matrix reduces its size to less than 160MB — a 10-fold reduction. This optimization is crucial when working with large-scale scRNA-seq datasets.

```
In [ ]: # Convert the sc object, a sparse matrix from a scRNA-seq dataset, into a dense
dense.size <- object.size(as.matrix(sc))

# Calculates the size of the sc object without converting it to a dense matrix.
sparse.size <- object.size(sc)

# Returns the size of the dense matrix and sparse matrix in MB.
format(dense.size, "MB")
format(sparse.size, "MB")

# Calculates the ratio of the size of the dense matrix to the size of the sparse
dense.size/sparse.size
```

We use the usual matrix commands to handle a sparse matrix. See below how to:

Know the dimensions of the matrix; Identify the class of a sparse matrix; Select rows and columns of a sparse matrix using numeric indices; Extract the names of the columns (or of the rows); Select rows (or columns) of a sparse matrix using the names of the rows (or columns)

```
In [ ]: # How many features?
# How many cells?
dim(sc)

# What's the class: sparse matrix
class(sc)

# Select some rows and some columns
# Is there anything special about this matrix?
sc[33495:33500,1:3]

# Select using rownames (colnames)
sc[c("MIR1302-2HG", "FAM138A", "OR4F5"), 1:30]

# What are the names of genes
rownames(sc)[1:3]

# Extract the names of cells
colnames(sc)[1:3]
```

How big is this count table?

```
In [ ]: # Show the dimensions of sc
print(dim(sc))

# Show the class of sc
print(class(sc))
```

Quality control

We will now create a Seurat object to continue with the analysis. In this step, we will apply some filters:

1. Remove genes that are detected in fewer than 3 cells.

2. Remove cells that express fewer than 200 genes.

These filtering steps help clean the data by excluding genes and cells with minimal expression, which might be uninformative. However, these filters are optional and you can adjust the thresholds or skip this step entirely if preferred.

```
In [ ]: sampleA <- CreateSeuratObject(counts = sc, project="sampleA",  
                                     min.cells=3, min.features=200)
```

```
In [ ]: # QC metrics for the first 5 cells  
head(sampleA@meta.data, 5)
```

What do we know about the sampleA object?

How many genes and how many cells does the Seurat object have?

Also, why are these numbers different?

Remember that we created the Seurat object specifying that each gene had to be present in, at least, 3 cells (otherwise, the gene is removed). We also specified that each cell is required to have, at least, 200 genes.

```
In [ ]: sampleA
```

```
In [ ]: ## Let's check the dimensions of the original matrix (sc)  
## and compared to the Seurat object (sampleA)  
dim(sc)  
dim(sampleA)  
dim(sc) - dim(sampleA)
```

What is the size of our Seurat object? How many genes and cells were removed from original table?

Identifying Doublets

Doublets occur when two or more cells are mistakenly captured together in the same well, which is a common issue in single-cell RNA sequencing. This can happen when the cell suspension is too concentrated.

To address this problem, we will use the `scDb1Finder` method to identify and remove doublets from our dataset. This method requires an object of class `SingleCellExperiment`, which is part of the Bioconductor infrastructure.

Steps:

1. Prepare the `SingleCellExperiment` object: This is necessary before applying the `scDb1Finder` method.
2. Apply `scDb1Finder`: The method uses a probabilistic approach to detect doublets, so results may vary slightly between runs.
3. Ensure reproducibility: To avoid variability in results, we will set a random seed using the `set.seed()` function. This ensures that the method produces the same results

each time it is run.

By setting the seed, we make the process reproducible, helping to confirm that the results are consistent across different runs.

```
In [ ]: sce <- as.SingleCellExperiment(sampleA)
sce

# We need to set.seed() because the scDblFinder command uses a probabilist strat
# This means that, everytime we run the command, it will produce results that ar
# The set.seed() comman will guarantee the same results everytime.

set.seed(123)
results <- scDblFinder(sce, returnType = 'table') %>%
  as.data.frame() %>%
  filter(type == 'real')

# %>% is the pipe operator from the tidyverse package.
# It allows you to pass the output of one function as the input to the next func

# Show the firsts rows of the results data frame
head(results)
```

Note that the results table includes a column called `class`, which categorizes cells into two types: singlet and doublet. As shown below, doublet cells are the ones that should be removed from the dataset before proceeding with further analysis. In other words, we want to keep only the cells labeled as singlets, which represent individual cells, and remove the doublets.

```
In [ ]: results %>%
  dplyr::count(class)
```

Save the results of `scDblFinder()` to reuse later

```
In [ ]: # This function is used to create a file path in a robust manner that is operati
# (works on Windows, Linux, etc.)
outfile = file.path('dataset1_control.txt')

# This function writes the data from a data frame or matrix to a text file.
write.table(results, outfile, sep='\t', quote=F,
            col.names=TRUE, row.names=TRUE)
```

Let's find the doublets and remove them from our matrix, i.e., let's keep only the singlets.

Remember that, in this session, we are focusing on the Seurat object (`sampleA`), and that's why we will subset this object.

```
In [ ]: keep = results %>%
  dplyr::filter(class == "singlet") %>%
  rownames()

# We are subsetting the Seurat object 'sampleA' to retain only the cells
# classified as 'singlet' based on the 'keep' vector.
sampleA <- sampleA[, keep]
sampleA
```

The percentage of mitochondrial reads is not automatically calculated by Cell Ranger. So, first we have to do this. We use this opportunity to include the percentage of MT reads as metadata to the Seurat object (sampleA). Remember that high percentage of mitochondrial reads is generally associated to high-stress and low-quality cells.

```
In [ ]: # Recognize mitochondrial genes and
# calculate the % of mitochondrial gene expression for each cell
# Then insert mitochondrial gene percentage into the metadata of the Seurat object
sampleA[["percent.mt"]] <- PercentageFeatureSet(sampleA, pattern="^MT-")
```

This function allows you to easily calculate the percentage of counts that belong to a specific subset of features for each cell. For example, you can use it to calculate the percentage of transcripts that map to mitochondrial genes. The calculation is done by summing the counts (UMIs) for the features in the subset (e.g., mitochondrial genes) and dividing this sum by the total sum of counts for all features (all genes) in each cell. The result is then multiplied by 100 to get the percentage.

In this example, the function adds a new column called percent.mt to the metadata associated with the sample (here, sampleA). This column contains the calculated percentage for each cell.

Terminology Clarification:

Feature: Refers to genes in this context.

Count: Refers to the UMI counts (Unique Molecular Identifiers) for each gene.

```
In [ ]: sampleA@meta.data %>% head()
```

The code provided is used to generate a violin plot for visualizing the distribution of certain features in the dataset using the Seurat package in R.

```
In [ ]: #This function is used to create violin plots, which display the distribution of
scPlot <- VlnPlot(sampleA, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"))
scPlot
#ggsave("01-VlnPlot.png", plot = scPlot, bg = 'white')
```

One dimensional violin plots are often difficult to interpret the entire distribution so let's try some histograms.

This is also a demonstration of how to make plots without using Seurat's visualization commands


```
In [ ]: # FetchData extract specific metadata or expression data from a Seurat object.
sampleA.qc <- FetchData(sampleA,
                        vars=c("nFeature_RNA", "nCount_RNA", "percent.mt"))

# Plot nCount_RNA histogram
scPlot <- sampleA.qc %>%
  ggplot() +
  geom_histogram(aes(x=nCount_RNA), bins=100)
scPlot
#ggsave("2-histogram.png", plot = scPlot, bg = 'white')
```

Try to extract the mean of nCounts_RNA

```
In [ ]: summary(sampleA.qc$nCount_RNA)
```

We see very clearly that the typical cell has about 20000 UMIs but this ranges from just a couple tens to over 60000

Let's zoom in on the low count end of the spectrum just to see if we can identify any structure (eg bimodality)

```
In [ ]: scPlot <- sampleA.qc %>%
  ggplot() +
  geom_histogram(aes(x=nCount_RNA), bins=100) +
  xlim(0,1500)
scPlot
#ggsave("03-histogram.png", plot = scPlot, bg = 'white')
```

Given the large dynamic range, perhaps a log scale will be useful

```
In [ ]: scPlot <- sampleA.qc %>%
  ggplot() +
  geom_histogram(aes(x=nCount_RNA), bins=200) +
  scale_x_log10()
scPlot
#ggsave("04-histogram.png", plot = scPlot, bg = 'white')
```

Now let's look at the median number of genes per cell

```
In [ ]: median(sampleA.qc$nFeature_RNA)
```

```
In [ ]: # Plot nFeature_RNA histogram with a vertical line at 3096
scPlot <- sampleA.qc %>%
  ggplot() +
  geom_histogram(aes(x=nFeature_RNA), bins=200) +
  geom_vline(xintercept = 3096, color="red")
scPlot
#ggsave("05-histogram.png", plot = scPlot, bg = 'white')

# Plot nFeature_RNA histogram with a vertical line at 10^(3.491)
scPlot <- sampleA.qc %>%
  ggplot() +
  geom_histogram(aes(x=nFeature_RNA), bins=200) +
  geom_vline(xintercept = 10^(3.491), color="red") +
  scale_x_log10()
scPlot
```

```
#ggsave("06-histogram.png",plot = scPlot, bg = 'white')

# Summary statistics of nFeature_RNA
summary(sampleA.qc$nFeature_RNA)
```

Looks like the typical cell has data on about 3000 genes, ranging from 200 (the minimum we set when we created the Seurat object) to over 9000

And how about the percentage mitochondrial reads?

```
In [ ]: scPlot <- sampleA.qc %>%
  ggplot() +
  geom_histogram(aes(x=percent.mt), bins=100) +
  geom_vline(xintercept = 10,color="red")

scPlot
#ggsave("07-histogram.png",plot = scPlot, bg = 'white')
summary(sampleA.qc$percent.mt)
```

The majority of cells are below **5%** but some are over **50%**

It is often more useful to look at multiple QC metrics together instead of individually. Lets try some simple 2D scatter plots

```
In [ ]: scPlot <- FeatureScatter(sampleA, feature1="nCount_RNA", feature2="percent.mt")
scPlot
#ggsave("08-FeatureScatter.png",plot = scPlot, bg = 'white')

In [ ]: scPlot <- FeatureScatter(sampleA, feature1="nCount_RNA", feature2="nFeature_RNA")
scPlot
#ggsave("09-FeatureScatter.png",plot = scPlot, bg = 'white')
```

Remember: we don't need to use only Seurat plotting tools.

`ggplot2` is a powerful visualization tool, which handles tables nicely to create plots. Check the plot above (nFeature vs. nCount) and consider that it may be of interest to understand how the data behaves as a function of the percentage of MT reads.

```
In [ ]: # Scatter plot of nCount_RNA vs nFeature_RNA colored by percent.mt
scPlot <- sampleA.qc %>%
  ggplot() +
  geom_point(aes(nCount_RNA, nFeature_RNA, colour=percent.mt), alpha=.50)
  scale_x_log10() +
  scale_y_log10()

scPlot
#ggsave("10-FeatureScatter.png",plot = scPlot, bg = 'white')
```

Most of this QC looks quite nice. We don't see strikingly different subpopulations of cells by these QC metrics. Just to be safe, let's remove cells with over 10% mitochondrial reads, which may be a sign of ex-vivo damage during sample handling and library generation

```
In [ ]: #Filter using more than one variable
sampleA <- subset(sampleA,
```

```
nCount_RNA > 500 & # Filter cells with more than 500 UMIs
nFeature_RNA < 7000 & # And filter cells with less than 7000 genes
percent.mt < 10) # And filter cells with less than 10% of m
```

Let's reuse the plot we created on the last exercise and create a new column (called keep) using the metadata table. This column will identify the cells that we removed above.

```
In [ ]: # This code has a if - else statement from the tidyverse package
# that creates a new column 'keep' based on conditions applied to existing columns
scPlot <- sampleA.qc %>%
  mutate(keep = if_else(nCount_RNA > 500 &
                        nFeature_RNA < 7000 &
                        percent.mt < 10, "keep", "remove")) %>%

  ggplot() +
  geom_point(aes(nCount_RNA, nFeature_RNA, colour=keep), alpha=.50) +
  scale_x_log10() +
  scale_y_log10()

scPlot
#ggsave("11-point.png", plot = scPlot, bg = 'white')
```

```
In [ ]: #sampleA <- subset(sampleA, subset = percent.mt < 10)
scPlot <- FeatureScatter(sampleA, feature1="nCount_RNA", feature2="percent.mt")
scPlot
#ggsave("12-FeatureScatter.png", plot = scPlot, bg = 'white')

print(dim(sc))
print(dim(sampleA))
```

Normalization

The variation in total RNA per cell can likely influence the results. This is something we might observe in our quality control (QC) steps, and since we haven't yet corrected for this, it's important to address this issue now.

A common and straightforward way to correct for this is to normalize the data. We will do this by dividing the counts for each gene by the total UMI (Unique Molecular Identifier) count for that specific cell. This step adjusts for the RNA capture efficiency that varies between cells, ensuring that expression levels are comparable across all cells, regardless of their total RNA content.

After normalization, we will apply a logarithmic transformation. This is done to make the data more manageable and reduce the impact of genes with very high expression, which can dominate the analysis.

Additionally, we will multiply the normalized counts by a scaling factor of 10,000 before applying the logarithmic transformation. This factor is somewhat arbitrary, but it is commonly used in practice. The scaling factor ensures that the resulting values are not too small and are easier to handle computationally. While the exact factor is not critical, using a standard value such as 10,000 makes the process consistent and allows for easier comparison between datasets.

But first, let's save an unnormalized version, which we'll use later for comparison

```
In [ ]: unnormalized <- sampleA
```

Now we can proceed:

```
In [ ]: sampleA <- NormalizeData(sampleA, normalization.method="LogNormalize",  
                                scale.factor=10000)
```

Identify variable genes

Let's start exploring some visualization for understand the patterns of gene expression change across cells

The first step in the analysis is to identify the genes that show the most variability across the dataset. Genes that do not vary much across cells are less likely to provide useful information for further analyses, so we will focus on the top 2000 most variable genes.

To do this, we will first apply a variance stabilizing transformation (vst). This transformation helps to adjust the relationship between the mean and variance of gene expression values, making it easier to compare genes with different levels of expression. The goal of vst is to reduce the effect of genes with high expression levels, which tend to have higher variability, and make the variability of genes more consistent across different expression levels. This step ensures that we can more accurately identify genes that truly vary across cells and are likely to be biologically significant.

```
In [ ]: sampleA <- FindVariableFeatures(sampleA, selection.method = "vst", nfeatures = 2000)
```

Let's see the names of the most highly variable genes now

```
In [ ]: top10 <- head(VariableFeatures(sampleA), 10)  
top10
```

Now, we go plot the variance (after `vst`) vs the mean of expression for each gene, coloring the top 2000 and labelling the top 10

```
In [ ]: plot1 <- VariableFeaturePlot(sampleA)  
  
scPlot <- LabelPoints(plot=plot1, points = top10, repel=T, xnudge=0, ynudge=0) +  
  theme(legend.position="none")  
scPlot  
#ggsave("13-VariableFeaturePlot.png", plot = scPlot, bg = 'white')
```

Scaling gene expression

The next step in the analysis is to scale the expression of each gene across cells. This means adjusting the expression values so that each gene has an average expression of 0

and a variance of 1. This scaling step is important because genes often have different expression levels (magnitude), and if we don't scale the data, genes with higher expression levels might dominate the analysis.

By scaling the expression, all genes will contribute equally to downstream analyses, regardless of their initial expression levels. This is a common technique used in many big data fields to ensure that features (like genes in this case) with different scales or magnitudes can be compared or used together effectively.

In Seurat, the scaling process does not overwrite the original, unscaled expression values. Instead, the scaled expression values are stored separately in the Seurat object under `sampleA[["RNA"]][@scale.data]`. This way, both the raw and scaled expression values are preserved, allowing you to use either version depending on the analysis or visualization you're performing.

scale the data

```
In [ ]: sampleA <- ScaleData(sampleA)
```

By default, only the most variable genes are scaled, which are used for further analysis such as dimensionality reduction.

```
In [ ]: sampleA[["RNA"]][@data[1:10,1:10]]
```

Dimension Reduction

PCA

We've identified the most variable genes and standardized their scales. Now, let's perform our first dimensionality reduction and visualization using PCA (Principal Component Analysis).

PCA is a technique used to reduce the number of dimensions (features) in a dataset while preserving as much variability (information) as possible. It does this by finding new variables, called principal components, that are combinations of the original features. These components capture the most important patterns in the data. In simple terms, PCA helps to simplify complex data by transforming it into a smaller set of meaningful components, making it easier to visualize and interpret.

Perform PCA

```
In [ ]: sampleA <- RunPCA(sampleA, features = VariableFeatures(sampleA))
scPlot <- DimPlot(sampleA, reduction="pca")
scPlot
#ggsave("14-DimPlot.png", plot = scPlot, bg = 'white')
```

Hmm, that doesn't show much structure. It seems like the vast majority of the variation across cells is coming from one direction in gene expression space. Maybe we can learn

something by looking at what genes contribute to that PC.

The explained variance by each principal component is the ratio between each eigenvalue and the sum of all eigenvalues. We need to remember that Seurat does not compute all the principal components, because it is computationally heavy. In our case, it calculated the first 50 PCs, so all the inference must be conditional on these 50 PCs (and not all the 3894 possible PCs).

Calculate the variance of each component

```
In [ ]: # Extract the PCA results from the Seurat object
pca <- sampleA[["pca"]]

# get the eigenvalues
# eigenvalues represent the amount of variance explained by each principal component
evs <- pca@stdev^2

# total variance means the overall variability present in the dataset before per
total.var <- pca@misc$total.variance

# Calculate the proportion of variance explained by each principal component
varExplained = evs/total.var

# Transform the PCA results into a data frame for visualization
pca.data <- data.frame(PC=factor(1:length(evs)),
                      percVar=varExplained*100)

# Calculate cumulative variance
pca.var <- cumsum(pca.data$percVar)

# Add cumulative variance to the data frame
pca.data$cumulVar <- pca.var
# pca.data$cumulVar = cumsum(pca.data$percVar) <- retire '#' para funcionar

# See the first 20 rows of pca.data
head(pca.data, 20)

# Plot variance explained by each principal component
scPlot <- pca.data[1:10,] %>%
  ggplot(aes(x=PC, y=percVar)) +
  geom_bar(stat='identity') +
  geom_hline(yintercept = 1, colour="red", linetype=3) +
  labs(title="Variance Explanation by PCA") +
  xlab("Principal Components") +
  ylab("Percentage of Explained Variance") +
  theme_bw()

scPlot
#ggsave("15-geom_bar.png",plot = scPlot, bg = 'white')

# Plot cumulative variance explained by each principal component
scPlot <- pca.data[1:10,] %>%
  ggplot(aes(x=PC, y=cumulVar)) +
  geom_bar(stat='identity') +
  geom_hline(yintercept = 50, colour="red", linetype=3) +
  labs(title="Cumulative Variance Explanation by PCA") +
  xlab("Principal Components") +
  ylab("Cumulative Percentage of Explained Variance") +
```

```

    theme_bw()
scPlot
#ggsave("16-geom_bar.png", plot = scPlot, bg = 'white')

```

Perhaps we can learn something by observing which genes contribute to those major components (PCs).

```

In [ ]: # Visualize PCA results with a heatmap for the first two principal components
#png("17-DimHeatmap.png", res= 300, height = 1920, width = 1920)
DimHeatmap(sampleA, dims = 1:2) #Select 2 PCs to visualize
#dev.off()

```

What do the other PCs look like?

```

In [ ]: #png("18-DimHeatmap.png", res= 300, height = 1920, width = 1920)
DimHeatmap(sampleA, dims = 1:5,      # Select 5 PCs to visualize
            cells = 500, balanced=T) # Select 500 cells to visualize
#dev.off()

```

Comparisons

How does normalization affect data? First, let's prepare the unnormalized data to the point where our dataset is located.

This code replaces the information layer that `ScaleData()` needs to **function** properly.

```

In [ ]: # Insert the unnormalized counts into the data slot of the RNA assay
unnormalized[["RNA"]]$data <- unnormalized[["RNA"]]$counts

In [ ]: unnormalized <- FindVariableFeatures(unnormalized, selection.method = "vst", nfe
unnormalized <- ScaleData(unnormalized)
unnormalized <- RunPCA(unnormalized, features = VariableFeatures(unnormalized))

In [ ]: NormalizeDataPlot <- FeaturePlot(sampleA, features="nCount_RNA")
UnnormPlot <- FeaturePlot(unnormalized, features="nCount_RNA")
options(repr.plot.width=12, repr.plot.height=6) # Change in the aspect ratio for
UnnormPlot | NormalizeDataPlot # See both plots side by side

```

What can we say about this figure?

Alternative Normalization

Satijalab developed its own normalization method implemented in Seurat, which they called 'sctransform'. See the original article: [Normalization and variance stabilization of single-cell RNA-seq data using regularized negative binomial regression](https://satijalab.org/seurat/articles/sctransform_vignette.html), which the authors describe as having a better capacity to correct for differences in sequencing depth.

Additional information is available at
https://satijalab.org/seurat/articles/sctransform_vignette.html

The following command performs the same functions as `NormalizeData()`, `ScaleData()`, and `FindVariableFeatures()`. It's worth noting that the method executes in this way when dealing with only one sample/individual; for multiple samples, some modifications are necessary depending on the experimental design.

See more in:

https://satijalab.org/seurat/archive/v4.3/sctransform_v2_vignette

```
In [ ]: BiocManager::install('glmGamPoi')
sampleA <- SCTransform(sampleA, verbose = T)
```

Now we can see that the tool's execution output tells us "Set default assay to SCT", which implies a new assay in the R object. This assay also contains 3 layers of information related to the expression (counts, data and scaled.data) just like the "RNA" assay we had been using, but transformed according to the `sctransform` method.

```
In [ ]: sampleA$RNA
sampleA$SCT
```

```
In [ ]: class(sampleA$RNA)
```

```
In [ ]: class(sampleA$SCT)
```

```
In [ ]: sampleA$RNA$data[1:5,1:20]
sampleA$SCT$data[1:5,1:20]
```

Currently the active Assay is "SCT" but you can reselect "RNA"

```
In [ ]: DefaultAssay(sampleA) <- "RNA"
sampleA
```

And return to "SCT"

```
In [ ]: DefaultAssay(sampleA) <- "SCT"
sampleA
```

This influences where Seurat's functions will look for data, although we can also define it within the functions themselves, as in `RunPCA()` where we explicitly use the "SCT" assay. This argument is `NULL` by default, which, as we mentioned, implies the use of the active assay. The reduction name was also modified to avoid overwriting the PCA generated with the "RNA" assay data.

You can review arguments for this and other functions on the Seurat website: <https://satijalab.org/seurat/reference/>

```
In [ ]: sampleA <- RunPCA(sampleA, features = VariableFeatures(sampleA),
                        assay = "SCT", reduction.name = "SCT.pca")
```

```
In [ ]: sctransform <- FeaturePlot(sampleA, features="nCount_RNA", reduction = "SCT.pca")
sctransform | NormalizeDataPlot
```


UMAP

UMAP (Uniform Manifold Approximation and Projection) is a nonlinear dimensionality reduction technique that is often used for visualizing complex datasets, such as single-cell RNA-seq data. Unlike PCA, which is a linear transformation, UMAP is capable of capturing more complex, nonlinear relationships between data points. This makes UMAP especially useful when the data has intricate structures that cannot be captured by linear methods.

The feature of UMAP is that it preserves both local and global structures in the data. This means that UMAP not only keeps the relationships between nearby points but also tries to maintain the broader overall structure of the dataset, which makes it great for visualization.

However, UMAP's bug or limitation is that it is computationally expensive and can be noisy when dealing with high-dimensional data directly. To address this, we often perform initial dimensionality reduction using PCA before applying UMAP. PCA reduces the number of dimensions by identifying the most important components in the data, which simplifies the structure and speeds up the UMAP calculations. Additionally, PCA can help remove some noise, improving the quality of the UMAP results.

In summary, UMAP is a powerful tool for visualizing complex data, and using it after PCA can make the process more efficient and produce clearer results.

```
In [ ]: # To use PCA for dimensionality reduction we have to choose how many principal c  
# Since PCA is Linear and orthogonal, the PC values are straightforward to inter  
# Let's Look at the top PCs.  
  
scPlot <- ElbowPlot(sampleA)  
options(repr.plot.width=6, repr.plot.height=6) # Change in the aspect ratio for  
scPlot  
  
#ggsave("23-ElbowPlot.png",plot = scPlot, bg = 'white')
```

Or we can see it generated for the data with scTransform

```
In [ ]: scPlot <- ElbowPlot(sampleA, reduction = "SCT.pca")  
scPlot
```

```
In [ ]: #By default we see the top 20 but we can ask for more if we like.  
scPlot <- ElbowPlot(sampleA, ndims=50)  
scPlot  
#ggsave("24-ElbowPlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: # Note that the previous RunPCA only calculated the top 50. If we want to Look  
# We could go back and re-run the PCA but for the sake of time Let's just use th  
  
# While there's no clear cutoff (there rarely is), it doesn't look like all top  
# Our calculations will of course be quickest if we only use 2 PCs, Let's see wh  
  
DefaultAssay(sampleA) <- "RNA" # Important: for simplicity, we will continue wor
```

```
sampleA <- RunUMAP(sampleA, dims=1:2, verbose=F)
scPlot <- DimPlot(sampleA, label=T) + NoLegend()
scPlot
#ggsave("25-DimPlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: # That seems suspicious.
# It doesn't at all match our expectations about gene expression
# profiles of PBMC subsets should look like.
# Let's see if it is a robust pattern, of if
# it changes a lot when we add just one more PC.
```

```
sampleA <- RunUMAP(sampleA, dims=1:3, verbose=F)
scPlot <- DimPlot(sampleA, label=T) + NoLegend()
scPlot
#ggsave("26-DimPlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: # Indeed, it changed quite a bit!
# It still doesn't really match what we might expect for PBMCs.
# Furthermore, the plot of PC values above doesn't plateau until somewhere
# in the 10-20 range. Let's use the top 15.
```

```
sampleA <- RunUMAP(sampleA, dims=1:15, verbose=F)
scPlot <- DimPlot(sampleA, label=T) + NoLegend()
scPlot
#ggsave("26-DimPlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: # Some nice clusters!
# First Let's make sure none of them are driven by QC artifacts.
```

```
scPlot <- FeaturePlot(sampleA, features=c("percent.mt"))
scPlot
#ggsave("25-FeaturePlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features=c("nCount_RNA"))
scPlot
#ggsave("26-FeaturePlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: # This looks good. It does not seem like either percent mitochondrial reads or
# While we're exploring, let's see about the top PCs.
# While both UMAP and PCA are in some sense trying to find natural variations in
# They are very different calculations in detail and we should not assume they a
scPlot <- FeaturePlot(sampleA, features=c("PC_1"))
scPlot
#ggsave("27-FeaturePlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: # PC1 does seem to be separating the cells on the far right from the others.
scPlot <- FeaturePlot(sampleA, features=c("PC_2"))
scPlot
#ggsave("28-FeaturePlot.png",plot = scPlot, bg = 'white')
```

Let's look at some individual genes. For an unsupervised approach, we could start with some of the most variable genes.

```
In [ ]: top10
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features=top10[1:4], ncol=2)
scPlot
#ggsave("29-FeaturePlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features=top10[5:8], ncol=2)
scPlot
#ggsave("30-FeaturePlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features=top10[9:10], ncol=1)
scPlot
#ggsave("31-FeaturePlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: # We can also look at some of the genes from top PCs
print(sampleA[["pca"]], dims=1:5, nfeatures=2)
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features=c("CTSC", "CD52", "LRRIQ1", "EFCAB1"))
scPlot
#ggsave("32-FeaturePlot.png",plot = scPlot, bg = 'white')
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features=c("CD68", "SERPING1", "IL32", "CD3E"))
scPlot
#ggsave("33-FeaturePlot.png",plot = scPlot, bg = 'white')
```

Clustering

Clustering is a method used in data analysis to group objects (such as cells, genes, or other entities) based on their similarities. In the context of scRNA-seq, clustering helps identify distinct cell populations by grouping cells with similar gene expression profiles. Now, let's perform formal clustering on this data. While there are various clustering algorithms available, we will use a nearest-neighbor graph approach combined with the Louvain algorithm to identify clusters or communities within the data.

This method works by building a graph where each data point (cell) is connected to its nearest neighbors, and then the Louvain algorithm is used to find groups of points that are densely connected to each other, indicating clusters.

One advantage of this approach is that it relies on distance metrics to the nearest neighbors, making it relatively robust to the curse of dimensionality. This means that it performs better than some other algorithms when working with high-dimensional data, such as single-cell RNA-seq data, where many variables can complicate clustering. Therefore, the nearest-neighbor graph and Louvain algorithm are a good choice for detecting meaningful clusters in complex datasets.

We'll calculate distances in the first 20 dimensions.

```
In [ ]: sampleA <- FindNeighbors(sampleA, dims=1:20)
sampleA <- FindClusters(sampleA)
```

Now we will visualize cluster membership in the UMAP space.

```
In [ ]: scPlot <- DimPlot(sampleA, reduction="umap")
scPlot
#ggsave("34-DimPlot.png", plot = scPlot, bg = 'white')
```

Almost all clustering algorithms have some sort of free parameter that controls how many clusters are identified.

In the Louvain algorithm we have the *resolution* which, holding all other parameters (such as the dimensions, the number of nearest neighbors, etc) constant, controls the number of clusters. Low (high) values for resolution give low (high) numbers of clusters.

```
In [ ]: # Let's explore this.
sampleA <- FindClusters(sampleA, resolution=0.01)
```

```
In [ ]: scPlot <- DimPlot(sampleA, reduction="umap")
scPlot
#ggsave("35-DimPlot.png", plot = scPlot, bg = 'white')
```

```
In [ ]: sampleA <- FindClusters(sampleA, resolution=10)
```

```
In [ ]: scPlot <- DimPlot(sampleA)
scPlot
#ggsave("36-DimPlot.png", plot = scPlot, bg = 'white')
```

It can be useful to see how clusters corresponding to one value of *resolution* correspond to those from another *resolution*. The *clustree* package does a nice job of visualizing this over the clusterings we have already performed.

```
In [ ]: head(sampleA[[ ]])
```

```
In [ ]: scPlot <- clustree(sampleA, prefix="RNA_snn_res.")
scPlot
#ggsave("37-clustree.png", plot = scPlot, bg = 'white', width = 16, height = 9)
```

```
In [ ]: sampleA <- FindClusters(sampleA, resolution=seq(0.1, 2, by=0.1))
```

```
In [ ]: scPlot <- clustree(sampleA, prefix="RNA_snn_res.")
scPlot
#ggsave("38-clustree.png", plot = scPlot, bg = 'white', width = 16, height = 9)
```

Now let's go back to the resolution that gave us three clusters

```
In [ ]: sampleA <- FindClusters(sampleA, resolution=0.01)
scPlot <- DimPlot(sampleA)
scPlot
#ggsave("39-DimPlot.png", plot = scPlot, bg = 'white')
```

Here is how we can identify marker genes for cluster 0

```
In [ ]: cluster0.markers <- FindMarkers(sampleA, ident.1=0, min.pct=0.25)
head(cluster0.markers)
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features="C1QA")
scPlot
#ggsave("40-FeaturePlot.png", plot = scPlot, bg = 'white')
```

Since we didn't specify, the previous calculation gave us genes either significantly over or under expressed in our population of interest. Sometimes we just want over expressed genes, which is easy to filter for.

```
In [ ]: cluster0.markers <- FindMarkers(sampleA, ident.1=0, min.pct=0.25, only.pos=T)
head(cluster0.markers, 10)
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features="RBP7")
scPlot
#ggsave("41-FeaturePlot.png", plot = scPlot, bg = 'white')
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features=c("C1QC"))
scPlot
#ggsave("42-FeaturePlot.png", plot = scPlot, bg = 'white')
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features=c("PTAFR"))
scPlot
#ggsave("43-FeaturePlot.png", plot = scPlot, bg = 'white')
```

```
In [ ]: # It's always good to try multiple different visualizations.
scPlot <- VlnPlot(sampleA, features=c("PTAFR", "C1QB", "RBP7"))
scPlot
#ggsave("44-VlnPlot.png", plot = scPlot, bg = 'white')
```

```
In [ ]: scPlot <- sampleA %>%
  FetchData(vars=c("C1QB", "seurat_clusters")) %>%
  ggplot() +
  geom_histogram(aes(x=C1QB), bins=100) +
  facet_wrap(. ~ seurat_clusters)
scPlot
#ggsave("45-histogram.png", plot = scPlot, bg = 'white')
```

```
In [ ]: # Now Let's find over-expressed marker genes for all clusters.
sampleA.markers = FindAllMarkers(sampleA, only.pos=TRUE,
                                min.pct=0.25, logfc.threshold=0.25)

# See the top markers for each cluster
head(sampleA.markers)

# Select the top 5 markers per cluster based on average Log2 fold change
topMarkers <- sampleA.markers %>%
  group_by(cluster) %>%
  top_n(n=5, wt=avg_log2FC)

# Heatmap of the top markers
scPlot <- DoHeatmap(sampleA, features = topMarkers$gene) + NoLegend()
scPlot
#ggsave("46-DoHeatmap.png", plot = scPlot, bg = 'white')
```

```
In [ ]: # If we crank up the resolution a little,
# we get some finer-grained separation of the UMAP clouds.
sampleA <- FindClusters(sampleA, resolution=0.2)
```

```
In [ ]: scPlot <- DimPlot(sampleA)
scPlot
#ggsave("47-DoHeatmap.png", plot = scPlot, bg = 'white')
```

Differential Expression

Differential expression analysis refers to the process of identifying genes that show significant differences in expression levels between different groups of cells or conditions. This analysis helps to uncover genes that are uniquely expressed in specific cell types, states, or in response to certain treatments or environmental factors.

```
In [ ]: # Let's see if cluster 2 and cluster 0 really have different gene expression
# patterns and, if so, whether those match any biology we already know and
# might expect to see in PBMCs.
cluster1vs0 <- FindMarkers(sampleA, ident.1 = 2, ident.2 = 0)
head(cluster1vs0, 20)
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features="MCEMP1")
scPlot
#ggsave("48-FeaturePlot.png", plot = scPlot, bg = 'white')
```

```
In [ ]: scPlot <- FeaturePlot(sampleA, features="CXCL11")
scPlot
#ggsave("49-FeaturePlot.png", plot = scPlot, bg = 'white')
```

```
In [ ]: # Note that CXCL11 is also expressed in another group. We had
# ignored them when looking for differentially expressed genes between clusters
# 2 and 0 so this should not be surprising.

# Separate only positive markers
sampleA.markers <- FindAllMarkers(sampleA, only.pos=TRUE, min.pct=0.25)

# Select the top 5 markers per cluster based on average log2 fold change
topMarkers <- sampleA.markers %>%
  group_by(cluster) %>%
  top_n(n=5, wt=avg_log2FC)

# Heatmap of the top markers
scPlot <- DoHeatmap(sampleA, features = topMarkers$gene) + NoLegend()
scPlot
#ggsave("50-DoHeatmap.png", plot = scPlot, bg = 'white')
```

```
In [ ]: topMarkers <- sampleA.markers %>%
  group_by(cluster) %>%
  top_n(n=5, wt=avg_log2FC)

# Heatmap of the top markers plus CXCL11
scPlot <- DoHeatmap(sampleA, features = c("CXCL11", topMarkers$gene)) + NoLegend()
scPlot
#ggsave("51-DoHeatmap.png", plot = scPlot, bg = 'white')
```

Annotating Cell Types

Interpreting the results from single-cell data analysis is often the most challenging part of the process. To gain a deeper understanding beyond just the clusters we identified, we can annotate cell types by comparing the expression profiles from our data to a reference dataset. This approach allows us to provide more meaningful interpretations of the clusters.

Best practices recommend a hybrid approach, in which automated annotation is used as a starting point, followed by manual validation based on canonical markers and biological consistency. Using databases like [PanglaoDB](#) and [CELLxGENE](#) can also be a useful alternative at this stage.

For more information, please refer to our Module 01.

Automated annotation

To start, we will load a reference dataset that includes the cell types we expect to encounter in our samples. The `celldex` package offers several well-curated and well-annotated reference datasets. Although most of these datasets originate from bulk RNA-Seq and microarray experiments, they are still very useful for annotating single-cell datasets. In our case, we will use the Blueprint and ENCODE reference datasets, which are commonly used for such purposes. These datasets will help us match the gene expression profiles from our single-cell data to known cell types, enhancing our ability to interpret the clusters meaningfully.

In R widely used tools include [SingleR](#), [Azimuth](#), and [scmap](#), many of which integrate naturally with Seurat or the Bioconductor ecosystem.

See the more informations in:

- [Reference-based analysis of lung single-cell sequencing reveals a transitional profibrotic macrophage](#)
- [Azimuth HuBMAP Consortium](#)
- [scmap: projection of single-cell RNA-seq data across data sets](#)

```
In [ ]: library(celldex)
ref <- BlueprintEncodeData()
ref
```

Then, to use the `SingleR()` function, we need to convert the `Seurat` object to a `SingleCellExperiment` Bioconductor object. The `SingleR()` function takes our dataset, the reference dataset and the labels of the reference dataset.

```
In [ ]: library(SingleR)

# Convert Seurat object to SingleCellExperiment object
```

```
my.sce <- as.SingleCellExperiment(sampleA)

# Execute SingleR to annotate cell types
pred <- SingleR(my.sce, ref=ref, labels=ref$label.main)

# See the distribution of predicted cell types
table(pred$labels)
```

We can use a heatmap to visualize the resulting cell type annotation. At the top of the heatmap, we see the color code identifying the different cell types

```
In [ ]: #png("52-plotScoreHeatmap.png", res = 300, width = 1920, height = 1920)
        plotScoreHeatmap(pred)
        #dev.off()
```

A heatmap can also be used to show the distribution of the cell types by the clusters we identified using `Seurat`.

```
In [ ]: # Create a contingency table comparing assigned labels to Seurat clusters
my.table <- table(Assigned = pred$pruned.labels,
                  cluster = my.sce$seurat_clusters)

my.table

# Plot heatmap of the contingency table
library(pheatmap)
pheatmap(log2(my.table + 1), filename = "53-pheatmap.png")
```

Manual annotation

This approach is based on set of known gene markers associated with a cell type. In this step, a robust set of marker genes and prior knowledge (based on literature or specialist, e.g pathologist, immunologist), as well annotation experience is ideal to work well. However this step is critical to risk of unclear cell type annotations, incomplete detection of gene marker during sequencing and subjectivity.

There are ways to work with manual annotations, including:

- Annotation based on a set of marker genes from the cell populations that we expect to find in our data;
- Identifying the marker genes in each cluster to infer which cell type represents that population.

As our dataset is represented by bronchoalveolar immune cells in patients with COVID-19. Then, we expect to find immune cells in our clusters.

```
In [ ]: scPlot <- DimPlot(sampleA, label = TRUE) + NoLegend()
        scPlot
```

Let's create a panel of known markers for cell populations that we expect to find in our dataset.


```

In [ ]: # Select a set of marker genes for different cell types.
# Create a list where each element corresponds to a cell population marker genes

genes_markers = list(
  Fibroblast = c(
    "LSAMP", "LAMA2", "CLMP", "CLIC2", "ABCA1", "RUNX1T1", "ABCC3", "ZEB1",
    "FBLN2", "ZFP36", "BNC2", "COL3A1", "PDGFRA", "C1QC", "CDH11", "NAV3"
  ),
  Epithelial = c(
    "CALN1", "MUC6", "CFTR", "SLC4A4", "PKHD1", "ANXA4", "GMDS", "HNF1B",
    "HNF4A", "MUC5B", "MUC5AC", "PLAC8L1", "SLC13A1", "FRAS1", "CHST9"
  ),
  Muscle = c(
    "MYH11", "TPM1", "LMOD1", "RYR2", "CARMN", "SOX5", "DMD", "RBPMS",
    "PTN", "TTN", "CALD1", "PRKG1", "CACNA1C"
  ),
  Immune = c(
    "DOCK2", "ELMO1", "MRC1", "MS4A6A", "PTPRC", "IKZF1", "THEMIS",
    "ZEB2", "ARHGAP15", "F13A1", "HSP90AA1"
  ),
  Neural = c(
    "NRXN1", "NCAM1", "GRIK2", "SORCS1", "CADM2", "PPP2R2B", "SCN7A",
    "FRMD5", "NRXN3", "ZNF536"
  ),
  Endothelial = c(
    "TFPI", "FLRT2", "VAV3", "CD36", "UTRN", "SMAD1", "MMRN1", "TSPAN5", "ZNF521"
  )
)

# Note that many of these markers may not appear in our datasets, however
# the combination of these markers allows for more safe annotation.

# Crate Dotplot

scPlot <- DotPlot( object = sampleA,
  features = genes_markers,
  group.by = "seurat_clusters")

scPlot$data <- scPlot$data %>%
  dplyr::filter(pct.exp >= 5) # Filter out genes with more than 5% in each clust

scPlot <- scPlot +
  scale_color_gradient2(
    low = "#2c7bb6", # Assign a color scale to the dotplot in different levels
    mid = "#b2182b",
    high = "#d7191c") +
  theme_bw(base_size = 12) + # Change non-data graph display
  theme(
    axis.text.x = element_text(angle = 90) # Change axis x angle
  )

# Change in the aspect ratio for a better view of the dotplot
options(repr.plot.width=14, repr.plot.height=8)
scPlot

#ggsave("dotplot_markers_facets.png", plot = scPlot, width = 23, bg = 'white')

```

As we expected, our dataset represents a set of immune cells. We can further differentiate their cell populations.

```

In [ ]: # Select a set of marker genes for immune cells
genes_markers = list(
  Myeloid = c(
    "LYZ", "S100A8", "S100A9", "LGALS3", "CTSS", "MS4A7", "LST1"
  ),
  Lymph_T = c(
    "CD3D", "CD3E", "TRAC", "TRBC1", "IL7R", "LTB"
  ),
  Lymph_B = c(
    "MS4A1", "CD79A", "CD79B", "CD74", "HLA-DRA", "CD37", "CD19"
  ),
  Plasma = c(
    "MZB1", "XBP1", "JCHAIN", "SDC1", "TNFRSF17", "IGKC", "IGHG1"
  ),
  DC = c(
    "FCER1A", "CLEC10A", "CD1C", "ITGAX", "LILRA4", "IRF7"
  ),
  NK = c(
    "NKG7", "KLRD1", "GNLY", "PRF1", "TRDC"
  ),
  Erythrocytes = c(
    "HBB", "HBA1", "HBA2", "ALAS2", "SLC4A1", "AHSP", "GYPA"
  )
)

# Note that many of these markers may not appear in our datasets, however
# the combination of these markers allows for more safe annotation.

# Plot figure
scPlot <- DotPlot(
  object = sampleA,
  features = genes_markers,
  group.by = "seurat_clusters",
)

scPlot$data = scPlot$data %>% # Filter out genes with more than 5% in each cluster
  dplyr::filter(pct.exp >= 5)

scPlot = scPlot +
  scale_color_gradient2(
    low = "#2c7bb6",
    mid = "#b2182b", # Assign a color scale to the dotplot in different levels
    high = "#d7191c" +
    theme_bw(base_size = 12) + # Change non-data graph display
    theme(
      axis.text.x = element_text(angle = 90) # Change axis x angle
    )
)

options(repr.plot.width=14, repr.plot.height=8)
scPlot

#ggsave("dotplot_immuneMarkers_facets.png", width = 23, plot = scPlot, bg = 'white')

```

```

In [ ]: # It is often useful to visualize the top differentially expressed genes for each cluster
clt_markers <- sampleA.markers %>%
  group_by(cluster) %>%
  filter(cluster == "0") %>% # Modify the cluster number you plan to annotate.
  top_n(n=6, wt=avg_log2FC)

```

```
options(repr.plot.width=15, repr.plot.height=10)

FeaturePlot(sampleA, features = clt_markers$gene, ncol = 3,
            col = c("lightgray", "#d7191c"), min.cutoff = "q10")

VlnPlot(sampleA, features = clt_markers$gene, ncol = 3, pt.size = 1)
```

Differential Abundance

In addition to changes in gene expression, alterations in cellular composition between experimental conditions (treated vs. control) represent an important mechanism to be explored in scRNA-seq studies. Differential abundance analysis aims to identify whether certain cell populations are enriched or not in a given group of interest, as implemented by the [MiloR](#) package.

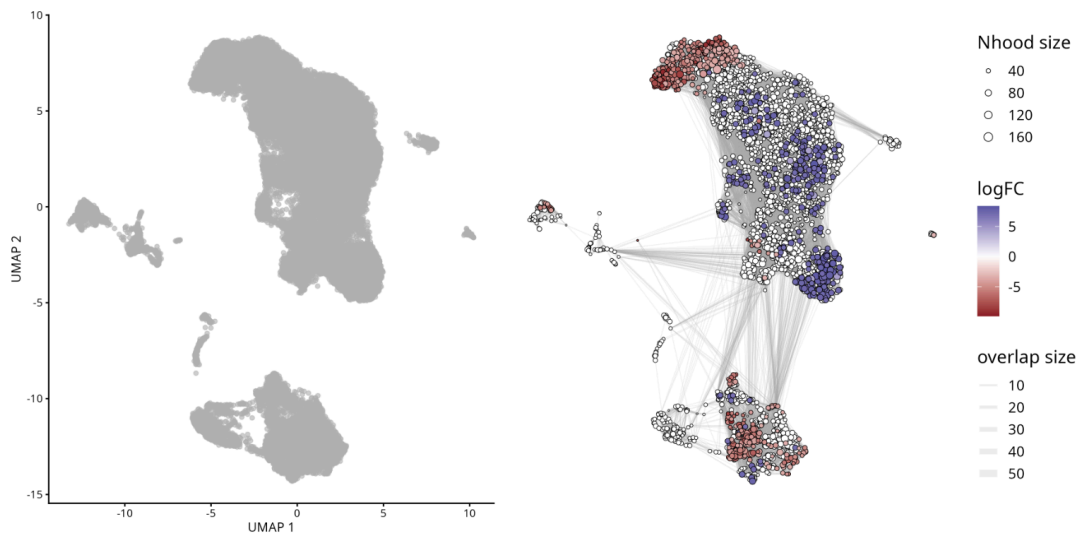
See original article: [Differential abundance testing on single-cell data using k-nearest neighbor graphs](#)

Changes in abundance may occur even in the absence of changes in gene expression. MiloR infers differential abundance using KNNs, which is appropriate for detecting spatial variations in cell density associated with conditions, avoiding exclusive reliance on rigid clustering in isolation.

Milo is a tool for analysis of complex single cell datasets and differential experimental conditions. While differential abundance is commonly quantified in discrete cell clusters, Milo uses partially overlapping neighbourhoods of cells on a KNN graph.

Milo analysis consists of 4 steps:

1. Prepare the `SingleCellExperiment` object: This is necessary before applying the MiloR method.
2. Sampling of representative neighbourhoods.
3. Testing for differential abundance of conditions in all neighbourhoods.
4. Accounting for multiple hypothesis testing using a weighted FDR procedure that accounts for the overlap of neighbourhoods.



Do not run in Colab!

To didactic purposes in this step, we will download other samples from our dataset to calculate the differential abundance of cells between mild and severe COVID-19 patients. To avoid making this tutorial too lengthy, we will simplify some steps.

However, we strongly recommend that you follow the best practice guidelines developed in SampleA object for the other samples.

```
In [ ]: # Below is the code to process and merge multiple scRNA-seq samples into a single
# we create a list to store individual Seurat objects for each sample
# and then merge them into a single Seurat object.
# To create this, we will loop through a list of filenames, read each file,
# create a Seurat object, perform quality control filtering, and store the Seurat

# A loop means that we will repeat a set of commands multiple times,
# one time for each item in a list or a vector.

# List of files to process
files <- c(
  "GSM4339769_C141_filtered_feature_bc_matrix.h5",
  "GSM4339770_C142_filtered_feature_bc_matrix.h5",
  "GSM4339771_C143_filtered_feature_bc_matrix.h5",
  "GSM4339773_C145_filtered_feature_bc_matrix.h5"
)

# Initialize an empty list to store Seurat objects
sc_list <- list()

for (i in files){
  sample_name <- strsplit(i, "_")[[1]][2] # Extract sample name from the file
  sc_data <- Read10X_h5(paste0("scRNAseq/", i)) # Read the 10X data
  sc_seurat <- CreateSeuratObject(counts = sc_data, project=sample_name, min.c

  sc_seurat[["percent.mt"]] <- PercentageFeatureSet(sc_seurat, pattern="^MT-")

  sc_seurat <- subset(sc_seurat, nCount_RNA > 500 & nFeature_RNA < 7000 & perc
```

```

    sc_list[[sample_name]] <- sc_seurat # Store the Seurat object in the list wi
}

# Merge all Seurat objects in the list into a single Seurat object
# x means the first Seurat object in the list
# y means the rest of Seurat objects in the list
# add.cell.ids assigns unique prefixes to cell barcodes from each sample to avoi
sc_merged <- merge(
  x = sc_list[[1]], # sample 1
  y = c(sc_list[[2]], sc_list[[3]], sc_list[[4]]), # sample 2, 3, and 4
  add.cell.ids = names(sc_list) # Use sample names as cell ID prefixes
)

# Display the first few rows of the merged Seurat object metadata
head(sc_merged)

```

```

In [ ]: # Add a new metadata column 'condition' based on the sample identifiers
# A if-else statement is used to classify samples C141 and C142 as 'mild' and ot
sc_merged$condition <- ifelse(grepl("^(C141|C142)", sc_merged$orig.ident), "mild
sc_merged

# Check the number of cells from each sample
table(sc_merged$orig.ident)
table(sc_merged$condition)

```

```

In [ ]: # Normalize and identify variable features for the merged dataset
sc_merged <- FindVariableFeatures(sc_merged, selection.method = "vst", nfeatures

# Run the standard workflow for visualization and clustering
sc_merged <- NormalizeData(sc_merged, verbose = FALSE)
all.genes <- rownames(sc_merged)
sc_merged <- ScaleData(sc_merged, features = all.genes, verbose = FALSE)
sc_merged <- RunPCA(sc_merged, features = VariableFeatures(sc_merged), verbose =
sc_merged <- FindNeighbors(sc_merged, dims=1:30, verbose = FALSE)
sc_merged <- FindClusters(sc_merged, resolution=0.8, verbose = FALSE)

# Run UMAP for dimensionality reduction
sc_merged <- RunUMAP(sc_merged, dims=1:30, verbose = FALSE)

```

Harmony was applied to correct for batch effects using the `orig.ident` metadata column. This software enables the integration of multiple datasets and effectively removes batch effects in single-cell RNA-seq analysis. For a detailed walkthrough of integration strategies, see **Module 05**.

More information about Harmony can be found at the official Broad Institute portal: [Harmony](#)

```

In [ ]: # Join layers before running Harmony Integration
sc_merged[["RNA"]] <- JoinLayers(sc_merged[["RNA"]], verbose = FALSE)

# Run Harmony for integration
sc_merged <- harmony::RunHarmony(sc_merged,
                                "orig.ident", # correct for batch effects based
                                reduction.save = "pca", # save new PCA reduction
                                verbose = FALSE)

# Find neighbors based on Harmony PCA

```

```

sc_merged <- FindNeighbors(sc_merged, dims = 1:30, verbose = FALSE)

# Find clusters in 0.8 resolution
sc_merged <- FindClusters(sc_merged, resolution = 0.8, verbose = FALSE)

# Run UMAP on Harmony embeddings
sc_merged <- RunUMAP(sc_merged, dims = 1:30, verbose = FALSE)

```

```

In [ ]: # Dimplot colored by clusters and samples
DimPlot(
  sc_merged,
  group.by = c("seurat_clusters", "orig.ident")
)

#ggsave("UMAP_COVID19_Clusters_Samples.png", width = 10, height = 5, bg = 'white'

```

```

In [ ]: # Load miloR package
library(miloR)

# Convert Seurat object to SingleCellExperiment
sce <- as.SingleCellExperiment(sc_merged)

# Create a Milo object from the SingleCellExperiment
milo <- miloR::Milo(sce)
milo

```

```

In [ ]: # Build the kNN graph for milo object
milo <- buildGraph(milo,
  k = 20, # number of nearest neighbors to use in our dataset
  d = 30, # number of dimensions to use in our dataset
  reduced.dim = "PCA") # reduced dimension to use

```

```

In [ ]: # Define neighborhoods in the milo object
milo <- makeNhoods(milo,
  prop = 0.1, # proportion of cells to include in each neighbo
  knn = 20, # number of nearest neighbors to use in our data
  d = 30, # number of dimensions to use in our dataset
  refined = TRUE) # whether to refine neighborhoods

```

```

In [ ]: # Plot neighborhood size histogram
plotNhhoodSizeHist(milo)

```

```

In [ ]: # Count cells in each neighborhood, adding metadata from the original Seurat obj
milo <- countCells(milo, meta.data = as.data.frame(colData(milo)),
  sample = "orig.ident") # specify the sample identifier column

# Display the first few rows of neighborhood counts
head(nhoodCounts(milo))

```

```

In [ ]: # Create a design data frame for differential abundance testing
# design data frame contains sample identifiers and their corresponding conditio
# and select relevant columns
milo_design <- data.frame(colData(milo))[,c("orig.ident", "condition")]

# convert condition to factor (mild is a baseline)
# set factor levels to ensure 'mild' is the baseline
milo_design$condition <- factor(milo_design$condition, levels = c("mild", "sever

```

```
# Remove duplicate rows to ensure unique sample-condition pairs
milo_design <- dplyr::distinct(milo_design)

# set row names to sample identifiers
rownames(milo_design) <- milo_design$orig.ident

# Return the milo_design data frame
milo_design
```

```
In [ ]: # Perform differential abundance testing using miloR
milo <- calcNhooDistance(milo,
                        d=30) # calculate neighborhood distances
```

```
In [ ]: # ensure row names match sample identifiers in milo object
rownames(milo_design) <- milo_design$orig.ident

# test for differential abundance
da_results <- testNhooDs(milo, design = ~ condition, design.df = milo_design)

# Display the first few rows of differential abundance results
da_results %>%
  arrange(- SpatialFDR) %>%
  head()
```

```
In [ ]: # Build neighborhood graph for visualization
milo <- buildNhooGraph(milo)

# Plot UMAP with differential abundance results
plotUMAP(milo) +
plotNhooGraphDA(milo, da_results,
                alpha=0.05) +
  plot_layout(guides="collect")

#ggsave("plotUMAP_with_DA_results.png", plot = last_plot(), width = 8, height =
```

Understanding Key Terms in Abundance Differential Analysis

- **Fold Change (FC):** Represents the magnitude of difference in abundance between conditions or groups. In this context, FC indicates how much a particular cell population is enriched or depleted when comparing experimental conditions.
- **False Discovery Rate (FDR):** A statistical correction applied to p-values to account for multiple testing. It helps control the proportion of false positives, ensuring that significant results are more reliable.
- **Cluster:** Refers to a group of cells identified by unsupervised clustering methods. Each cluster represents a population with similar gene expression profiles, and abundance differential analysis tests whether the frequency of these clusters changes across conditions.

Together, these metrics allow us to interpret whether specific cell populations are significantly expanded or reduced, while maintaining statistical rigor and biological relevance.

GO Enrichment Analyses

Enrichment analysis is a powerful tool for extracting biologically relevant information from single-cell datasets, facilitating the discovery of key genes, pathways, and cellular processes. In scRNA-seq, researchers often deal with complex tissues composed of various cell types. This analysis can reveal novel biological insights by highlighting unexpected associations between gene sets and Gene Ontology (GO) terms, providing a deeper understanding of cellular functions and disease mechanisms.

GO Enrichment Analysis is a method used to identify overrepresented GO terms within a set of genes. This analysis helps to determine whether certain biological processes, molecular functions, or cellular components are significantly associated with the genes of interest.

Gene Ontology (GO) is a comprehensive system that categorizes genes into hierarchical categories based on:

- Biological Processes (BP): The biological objectives the gene contributes to, such as cell cycle, apoptosis, or metabolism.
- Molecular Functions (MF): The molecular activities performed by the gene product, such as binding or catalytic activity.
- Cellular Components (CC): The location within the cell where the gene product is active, such as the nucleus or mitochondrion.

Additionally, genes are commonly referenced using standardized identifiers:

- Gene Symbol: A short, human-readable abbreviation (e.g., TP53, ACTB) that represents the gene name.
- EntrezID: A unique numerical identifier assigned by NCBI's Entrez Gene database, ensuring consistency across datasets and tools.

```
In [ ]: #Filter genes
my.genes = sampleA.markers %>%
  filter(abs(avg_log2FC) > 1, # This condition retains rows where the absolute v
p_val_adj < 0.10) %>%      # This condition retains rows where the adjusted p
dplyr::select(gene) %>%    # The select function from the dplyr package is us
pull() # The pull function extracts the values from the selected gene column a

# Display the firsts few rows
head(my.genes)
```

```
In [ ]: nrow(sampleA.markers) # This function returns the number of rows
genes <- sampleA.markers %>% arrange(desc(avg_log2FC)) # Sort the data frame in
head(genes)
```



```
In [ ]: library(clusterProfiler) # Provides functions for biological term classification

#Convert Gene Identifiers
my.map = bitr(my.genes, # This function converts gene identifiers from one type
              fromType="SYMBOL", # Specifies that the input gene identifiers are
              toType="ENTREZID", # Specifies that the output should be Entrez ID
              OrgDb="org.Hs.eg.db") # Specifies the organism database for human

# Display the First Few Entries
head(my.map)

# Calculate the Number of Genes Not Mapped
length(my.genes) - nrow(my.map)
```

GO Classification

```
In [ ]: # subontologies: Biological Process (BP), Molecular Function (MF), Cellular Comp
library(org.Hs.eg.db) # Library, which contains annotations for human genes.

# groups genes based on their GO terms
ggo_bp = groupGO(gene = my.map$ENTREZID, # Specifies the list of gene ENTREZ ID
                 OrgDb = org.Hs.eg.db, # Specifies the organism database for
                 ont = "BP", # Specifies that the GO grouping should be bas
                 readable = TRUE) # Converts the Entrez IDs to gene symbols for

# displays the first few entries of the ggo_bp object
head(ggo_bp)
```

```
In [ ]: # subontologies: BP, MF, CC
ggo_mf = groupGO(gene = my.map$ENTREZID, # Specifies the list of gene ENTREZ ID
                 OrgDb = org.Hs.eg.db, # Specifies the organism database for
                 ont = "MF", # Specifies that the GO grouping should be bas
                 readable = TRUE) # Converts the Entrez IDs to gene symbols for

# displays the first few entries of the ggo_mf object
head(ggo_mf)
```

```
In [ ]: ## subontologies: BP, MF, CC
ggo_cc = groupGO(gene = my.map$ENTREZID, # Specifies the list of gene ENTREZ ID
                 OrgDb = org.Hs.eg.db, # Specifies the organism database for h
                 ont = "CC", # Specifies that the GO grouping should be bas
                 readable = TRUE) # Converts the Entrez IDs to gene symbols for

# displays the first few entries of the ggo_cc object
head(ggo_cc)
```

GO over-representation analysis

```
In [ ]: # This function performs GO enrichment analysis on a given set of genes.
ego <- enrichGO(gene = my.map$ENTREZID, # Specifies the list of gene ENTREZ ID
                OrgDb = org.Hs.eg.db, # Specifies the organism database for hum
                ont = "BP", # Specifies that the GO grouping should be based
                pAdjustMethod = "BH", # The method used for adjusting p-values t
                pvalueCutoff = 0.01, # The p-value threshold for significance
                qvalueCutoff = 0.05, # The q-value threshold for significance
                readable = TRUE) # Converts the Entrez IDs to gene symbols
```

```
# displays the first few entries of the ego object  
head(ego)
```

GO enrichment in another organisms

Importantly, pathway enrichment analysis is not restricted to human or animal models. For non-model organisms and plants, a wide range of dedicated databases and functional annotations are available, enabling robust interpretation of transcriptomic data across diverse species.

In plant research, enrichment analyses commonly rely on plant-specific pathway repositories and Gene Ontology annotations tailored to species such as *Arabidopsis thaliana*, rice, and maize. These resources are essential for investigating key biological processes, including plant development, stress and immune responses, hormone signaling, and metabolic adaptations to environmental cues.

From a technical perspective, any gene identifier type supported by an appropriate `OrgDb` object can be directly used in GO-based analyses within the Bioconductor ecosystem. This flexibility allows seamless functional annotation across multiple organisms and identifier systems.

Several curated pathway databases further expand the scope of enrichment analyses across taxa.

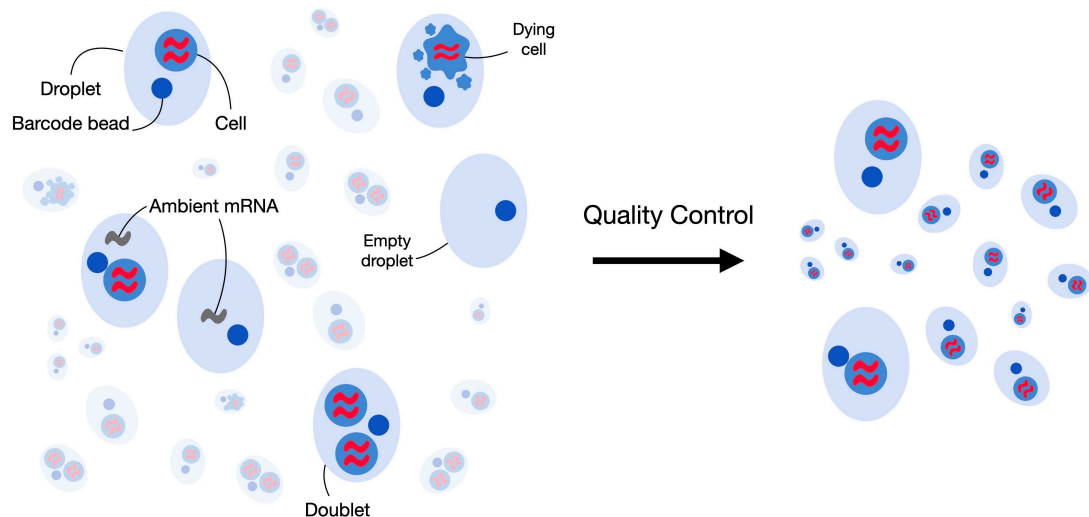
- [WikiPathways](#) is an open, community-driven platform dedicated to the curation of biological pathways, offering broad organismal coverage ranging from vertebrates to bacteria and plants. By adopting a collaborative curation model, WikiPathways complements established pathway resources such as `KEGG`, `Reactome`, and `Pathway Commons`.
- In the context of plant biology, [Plant Reactome](#) provides an open-source, open-access, manually curated and peer-reviewed pathway database encompassing more than 100 plant species.
 - For organisms lacking a dedicated `OrgDb` package in Bioconductor (e.g., species without an available annotation database analogous to `org.Mm.eg.db` in mouse)
 - Users can retrieve organism-specific annotations—when available—through online resources such as [AnnotationHub](#), ensuring that enrichment analyses remain accessible even for less-characterized species.

Ambient RNA Correction (Optional)

In some cases, there may be suspicion that the data contains contamination from cells that were destroyed and released their contents into the medium during library preparation, which could alter the RNA counts in intact cells. Tools, such as SoupX,

analyze the barcodes detected as background and their RNA content to identify transcripts potentially increased by contamination in the barcodes detected as cells and correct for their effect.

See the original article [SoupX removes ambient RNA contamination from droplet-based single-cell RNA sequencing data](#)



To show how simple this process can be, we will use a dataset generated with 10x technology and mapped with Cell Ranger (v8.0.1). This data corresponds to a sample from a head and neck cancer study.

Dataset available from NCBI/SRA: <https://www.ncbi.nlm.nih.gov/sra/?term=SRR10340945>.

The original article: [B cell signatures and tertiary lymphoid structures contribute to outcome in head and neck squamous cell carcinoma](#)

```
In [ ]: download.file("https://github.com/integrativebioinformatics/scNotebooks/blob/main/shell_call("tar -xvzf SRR10340945-Cellranger_outs_Folder.tar.gz")
shell_call("ls -lh")
```

As can be seen in the following code, for 10x data the process is as simple as loading the results folder obtained from cellranger and executing the `autoEstCont()` function.

```
In [ ]:
```

```
In [ ]: # Load SoupX and DropletUtils Libraries
library(SoupX)
library(DropletUtils)

# set a seed for reproducibility
set.seed(123)

# Load the 10X Genomics data using SoupX
sc = load10X("Mapped_SRR10340945/outs/")

# Execute automatic contamination estimation
```

```
#png("SoupX_SRR10340945.png", width = 1080, height = 1080)
sc = autoEstCont(sc)
#dev.off()
```

```
In [ ]: # Separate the estimated contamination fraction
rhoEst <- sc$fit$rhoEst

# Fraction of contamination estimated by SoupX
print(rhoEst)

# Normale, counts are integers numbers,
# but for compatibility DropletUtils requires rounded values
out = adjustCounts(sc, roundToInt = TRUE)

# For exporting the data in a folder in the CellRanger output format
DropletUtils::write10xCounts("./SRR10340945_clean", out)
```

```
In [ ]: shell_call("ls -lh SRR10340945_clean")
```

For data where the complete Cell Ranger output folder ("outs") is not accessible or where the data was not generated using 10x technology, SoupX can also be used by implementing a series of alternative steps. These steps involve generating a cluster, since the previous method uses the cluster generated by Cell Ranger, whose information is available in the "outs" folder, in addition to the account matrices.

In the following lines of code, you will see an example adapted from <https://cellgeni.github.io/notebooks/html/new-10kPBM-C-SoupX.html> with the data we just used.

It is essential to have both matrices: the filtered account matrix and the raw account matrix.

```
In [ ]: filt.matrix <- Read10X_h5("Mapped_SRR10340945/outs/SRR10340945.filtered_feature_
raw.matrix <- Read10X_h5("Mapped_SRR10340945/outs/SRR10340945.raw_feature_bc_ma
srat <- CreateSeuratObject(counts = filt.matrix)

# Create SoupChannel object from raw and filtered matrices
soup.channel <- SoupChannel(raw.matrix, filt.matrix)
soup.channel
```

Now we will execute the normalization steps up to the clustering required by SoupX. For simplicity and speed, we use `NormalizeData()`, but you can replace it with `SCTransform()` as implemented in the tutorial we adapted if you prefer.

```
In [ ]: srat <- NormalizeData(srat, verbose = F)
srat <- FindVariableFeatures(srat, verbose = F)
srat <- ScaleData(srat, verbose = F)
srat <- RunPCA(srat, verbose = F)
srat <- RunUMAP(srat, dims = 1:30, verbose = F)
srat <- FindNeighbors(srat, dims = 1:30, verbose = F)
srat <- FindClusters(srat, verbose = T)
```

Then we integrate the information obtained from the processing with seurat into the previously defined SoupChannel object

```
In [ ]: meta <- srat@meta.data
        umap <- srat@reductions$umap@cell.embeddings

        # Set cluster identities and UMAP coordinates in SoupChannel object
        soup.channel <- setClusters(soup.channel,
                                    setNames(meta$seurat_clusters, rownames(meta)))

        # Set UMAP coordinates in SoupChannel object
        soup.channel <- setDR(soup.channel, umap)
```

Finally, we can estimate the contamination and save our corrected data.

```
In [ ]: soup.channel <- autoEstCont(soup.channel)

        # Adjust counts to remove contamination
        adj.matrix <- adjustCounts(soup.channel, roundToInt = T)

        # Write the adjusted counts to a 10X format directory
        DropletUtils::write10xCounts("SRR10340945_cleanV2", adj.matrix)
```

Other tools that can be used to implement this environmental RNA correction process, and which you can find more information about if you are interested, are [DecontX](#) and [CellBender](#)